

UNIT 4

CONTROL STRUCTURE



STRUCTURES/STATEMENTS

CONTROL STRUCTURES/STATEMENTS

Control structures control the flow of execution in a program or function. Control structures are used to repeat any block of code, transfer control to a specific block of code, and make a choice by selection. There are three basic control structures in C++ programming.

Selection / Decision Making Control Structure

Loops / Iteration Control Structure

Jumps

SELECTION/DECISION MAKING CONTROL STRUCTURES/STATEMENTS:

The selection control structure allows a number of conditions which lead to a selection of one out of several alternatives. There are three types of selection control structure:

If selection structure / statement

If-else selection structure / statement

Switch selection structure / statement

if SELECTION STATEMENT:

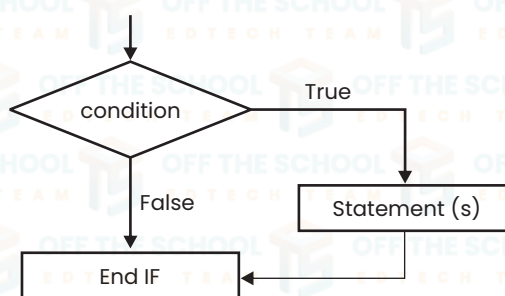
An if statement is a conditional statement that tests a particular condition. Whenever that condition evaluates as true, performs an action, but if it is not true, then the action is skipped.

SYNTAX

The general syntax of if statement is:

```
if (condition)
{
    Statement (s);
}
```

FLOW DIAGRAM



if SELECTION STATEMENT EXAMPLE

```
#include<iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter an integer number: ";
    cin >> num;
    if(num > 0)
    {
```

OUTPUT

Enter an integer number : 10
You entered a positive integer : 10

```

    cout << "You entered a positive integer: " << num << "\n";
}
return 0;
}

```

NESTED if STATEMENT

An if condition can be written as deeply as needed within the body of another statement. This is called a nested if statement.

The general syntax of nested if statement is:

```

if (condition1)
{
    if (condition2)
    {
        statements;
    }
}

```

NESTED if STATEMENT EXAMPLE

```

#include<iostream>
using namespace std;
int main()
{
    int exp, status;
    cout << "Enter experience: ";
    cin >> exp;
    cout << "\n Enter status: ";
    cin >> status;
    if (exp >= 4)
    {
        if (status >= 2)
        {
            cout << "\n Bonus Given to Employee" << "\n";
        }
    }
    return 0;
}

```

OUTPUT

```

Enter experience: 6
Enter status: 3
Bonus Given to Employee

```

if-else SELECTION STATEMENT

An if-else selection structure performs a certain action when the condition is true and some different action when the condition is false.

SYNTAX

The general syntax of if-else statement is:

```

if (condition)
{
    statement(s);
}

```



```
else
{
    statement(s);
}
```

if-else SELECTION STATEMENT EXAMPLE

```
#include<iostream>
using namespace std;
int main()
{
    int number;
    cout << "Enter an integer:";
    cin >> number;
    if (number >= 0)
    {
        cout << "The number is a positive integer:" << number << "\n";
    }
    else
    {
        cout << "The number is a negative integer:" << number << "\n";
    }
    cout << "This line is always printed.";
    return 0;
}
```

OUTPUT

Enter an integer: -5
 The number is a negative integer: -5
 This line is always printed.

else-if SELECTION STATEMENT:

Nested if-else statements test for multiple conditions by placing if-else statements inside if-else statements. When a condition is evaluated as true, the corresponding statements are executed and the rest of the structure is skipped. This structure is also referred to as the if-else-if ladder.

else-if SELECTION STATEMENT EXAMPLE

```
#include<iostream>
using namespace std;
int main()
{
    int per;
    cout << "\nEnter your percentage:";
    cin >> per;
    if (per >= 80)
    {
        cout << "Your grade is A+:";
    }
    else if (per >= 70)
    {
        cout << "Your grade is A:";
    }
    else if (per >= 60)
```

OUTPUT

Enter your percentage: 70
 Your grade is A:

```
{  
    cout << "Your grade is B:";  
}  
else if (per >= 50)  
{  
    cout << "Your grade is C:";  
}  
else  
{  
    cout << "Failed:";  
}  
return 0;  
}
```

SWITCH STATEMENT

Switch statement is a control statement that allows to select only one choice among the many given choices. The expression in switch evaluates to return an integer or character value, which is then compared to the values present in different cases. It executes that block of codes which matches the case value. If there is no match, then the default block is executed (if present).

SYNTAX

The general form of switch statement is,

switch(variable)

```
{  
    case constant 1:  
    {  
        statement (s);  
        break;  
    }  
    case constant 2:  
    {  
        statement (s);  
        break;  
    }  
    default:  
    {  
        statement (s);  
        break;  
    }  
}
```

SWITCH STATEMENT EXAMPLE:

```
#include<iostream>  
using namespace std;  
int main()  
{
```



```
char op;
float num1, num2;
cout << "Enter an operator (+, -, *, /):";
cin >> op;
cout << "Enter two numbers:" << "\n";
cin >> num1 >> num2;
switch (op)
{
    case '+':
        cout << num1 << "+" << num2 << "=" << num1 + num2;
        break;
    case '-':
        cout << num1 << "-" << num2 << "=" << num1 - num2;
        break;
    case '*':
        cout << num1 << "*" << num2 << "=" << num1 * num2;
        break;
    case '/':
        cout << num1 << "/" << num2 << "=" << num1 / num2;
        break;
    default:
        cout << "Error! The operator is not correct";
}
return 0;
}
```

OUTPUT

→ Enter an operator: +
 → Enter two numbers:
 → 10, 15
 → 10 + 15 = 25

DIFFERENCES BETWEEN if-else & SWITCH STATEMENT:

If-else statement is used to select among two alternatives. The switch statement is used to select among multiple alternatives.

If-else can have values based on constraints. Switch can have values based on user choice.

Float, double, char, int, and other data types can be used in if-else condition. Only int and char data types can be used in switch block.

It is difficult to edit the if-else statement, if the nested if-else statement is used. It is easy to edit switch cases as they are recognized easily.

LOOP/ITERATION CONTROL STRUCTURE:

Iteration or loop in computer programming is a process wherein a set of instructions or structures are repeated in a sequence a specified number of times until a condition is true. When the set of instructions is executed again, it is called an iteration. A loop statement allows us to execute a statement or a group of statements multiple times.

C++ provides the following types of loops to handle looping requirements:

- for loop
- while loop
- do-while loop

for LOOP:

A for loop is a repetition or iteration control structure that repeats a statement or block

of statements for a specified number of times. The for-loop statement includes the initialization of the counter, the condition, and the increment. The for loop is commonly used when the number of iterations is known.

SYNTAX

The general syntax of for loop is:

```
{
    statement(s);
}
```

for Loop Example

```
#include<iostream>
using namespace std;
int main ()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        cout << i << " ";
    }
    return 0;
}
```

OUTPUT

12345678910

While Loop

The while loop allows the programmer to specify that an action is to be repeated while some conditions remain true. It is used when the exact number of loop repetitions before the loop execution begins are not known.

SYNTAX

The general syntax of a while loop is:

```
while(condition)
{
    statement(s);
}
```

while Loop Example

```
#include<iostream>
using namespace std;
int main()
{
    int i = 1;
    while (i <= 10)
    {
        // The content inside the loop is missing from the extraction
    }
    cout << i << " ";
    i++;
}
```



```
}
return 0;
}
```

do-while loop

A do-while loop is similar to a while loop, except that a do-while loop is guaranteed to execute at least one time, even if the condition fails for the first time. Unlike for and while loops, which test the loop condition at the start of the loop, the do while loop checks its condition at the end of the loop.

SYNTAX

The general syntax of a do-while loop is:

```
do
{
    statement(s);
}
while(condition);
```

OUTPUT

12345678910

do while Loop Example

```
#include<iostream>
using namespace std;
int main()
{
    int i = 1;
    do
    {
        cout << i << " ";
        i++;
    }
    while (i <= 10);
    return 0;
}
```

Nested Loops

When one for, while or do while loop is nested within another loop, they are said to be nested. The inside loop is completely repeated for each repetition of the outside loop.

Nested Loop Example

```
#include<iostream>
using namespace std;
int main()
{
    int row, column;
    for (row = 1; row <= 5; row++)
    {
        for (column = 1; column <= 3; column++)
        {
            cout << " ";
        }
    }
}
```



```

    }
    // The content after this line is missing from the extraction
}
// There may be additional content here that is not included in the extraction.
}
cout << "\n";
}
return 0;
}

```

Differences Between for, while, AND do while loop

"for" Loop	"while" Loop	"do while" Loop
"for" loop is usually used in situations where we know at the start of loop that how many times loop block will execute. It is also called definite repetition loop.	It is usually used in situations where we do not know at the start of loop that how many times loop block will execute. It is also called indefinite repetition loop.	It is usually used in situations where we do not know at the start of loop that how many times loop block will execute. It is also called indefinite repetition loop.
It is called counter controlled loop as loop is controlled by a counter value, at each iteration counter value will increase or decrease.	It does not need a counter value for its execution.	It does not need a counter value for its execution.
It has three parts: first initialization, second condition testing, and third increment or decrement.	It has only one part which is condition testing. If needed, initialization is done before the loop and increment or decrement is done in loop body.	It has only one part which is condition testing. If needed, initialization is done before the loop and increment or decrement is done in loop body.
It is pre-test loop as condition is tested at start of loop.	It is pre-test loop as condition is tested at start of loop.	It is post-test loop as condition is tested at the end of loop. So this loop executes at least once.

Jump Statements:

These statements change the normal execution of program and jumps over the specific part of program.

Following are the jumps statements used in C++.

- break
- continue
- goto
- return
- exit ()

break STATEMENT:

The break statement allows you to exit a loop or a switch statement from any point within its body, bypassing its normal termination expression. It can be used within any C++ structure.

break EXAMPLE

```

#include <iostream>
using namespace std;

```

```
int main()
{
    int count;
    for(count = 1; count <= 100; count++)
    {
        cout << count;
        if(count = 10)
            break;
    }
    return 0;
}
```

continue STATEMENT:

The continue statement is just the opposite of the break statement. Continue forces the next iteration of the loop to take place, skipping the remaining statements of its body.

continue EXAMPLE

```
#include<iostream>
using namespace std;
int main()
{
}
{
    int count;
    for(count = 1; count <= 10; count++)
    {
        if(count = 5)
            continue;
        cout << count;
    }
    return 0;
}
```

goto STATEMENT:

In C++ programming, the goto statement is used for altering the normal sequence of program execution by transferring control to some other parts of the program.

goto EXAMPLE

```
#include<iostream>
using namespace std;
int main()
{
    float num, average, sum = 0.0;
    int l, n;
    cout << "Maximum number of inputs:";
    cin >> n;
    for(l = 1; l <= n; l++)
    {
```



```
cout << "Enter number" << i << " ";
cin >> num;
if (num < 0.0)
{
    // Control of the program move to jump:
    goto jump;
}
sum += num;
}
Jump:
    average = sum / n;
}
cout << "\n Average = " << average;
return 0;
}
```

return STATEMENT:

The return statement returns the flow of the execution to the function from where it is called. This statement does not mandatorily need any conditional statements. As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called.

SYNTAX:

return (expression / value);

exit() STATEMENT:

The exit function, declared in `<stdlib.h>`, terminates a C++ program. The value supplied as an argument to exit is returned to the operating system as the program's return code or exit code.

SYNTAX:

void exit (int);